- `reqWinner()`--This function counts the votes and identifies the winning proposal by its number. It executes the counting every time it's called. During testing, this is okay because you might call the function once or twice, but for production, you may want to optimize it. (Also, in future designs, you'll move this function off-chain or out of the smart contract code.) Note that this function is a "view" function, so it's not recorded on the chain.

Review the roles of these functions before you move on to testing the complete Ballot in the Remix IDE.

3.3 Testing

Testing of smart contracts is a crucial step in the Dapp design process. Chapter 10 is fully dedicated to writing automated test scripts. In this chapter, you'll start learning about the basics of testing to lay the foundation for test-driven development.

Load the code for the ballot problem into the Remix IDE in a file named BallotV3.sol, and compile it. Click the Deploy & Run Transactions icon, and within the JavaScript VM, choose an account address in the Account box in the panel at the top left. To the right of the Deploy button, you'll see a box for the number of proposals. Enter the number of proposals (such as 3) in the text box next to it, and click Deploy. This action will invoke the constructor with number of proposals (in this case 3) as a parameter.

By now, you should be familiar with the various areas of the Remix IDE. Figure 3.7 shows a screenshot taken during the testing of the Ballot smart contract. During testing, you'll be